PCET Education Trust

Pimpri Chinchwad College OF Engineering, Pune

Database Management Systems Lab

ASSIGNMENT NO. 03

Name	PRN
Atharva Lokhande	124B1F009

Batch A1

Guided By: Mrs. Alpana Borse Ma'am

Case Study: Airline Management System

SQL Queries:

Part A: Joins, Subqueries, and Views

1. Inner Join: This query retrieves booking details along with corresponding passenger information by joining the Bookings and Passengers tables.

```
mysql> SELECT
    ->     b.booking_id,
    ->     p.name AS passenger_name,
    ->     p.email,
    ->     b.booking_date
    -> FROM Bookings b
    -> INNER JOIN Passengers p
    -> ON b.passenger_id = p.passenger_id;
+------------+----------------+-----------------+---------------------+
| booking_id | passenger_name | email           | booking_date        |
+------------+----------------+-----------------+---------------------+
|          1 | Rahul Sharma   | rahul@gmail.com | 2026-04-10 12:00:00 |
|          2 | Priya Verma    | priya@gmail.com | 2026-04-11 14:30:00 |
|          3 | Amit Singh     | amit@gmail.com  | 2026-04-12 16:00:00 |
+------------+----------------+-----------------+---------------------+
3 rows in set (0.01 sec)
```

2. Left Join : It ensures every passenger is included, even if they have no bookings or tickets, with NULL values for missing data.

```
mysql> SELECT
    ->     p.name AS passenger_name,
    ->     b.booking_id,
    ->     t.ticket_id,
    ->     t.seat_number
    -> FROM Passengers p
    -> LEFT JOIN Bookings b
    -> ON p.passenger_id = b.passenger_id
    -> LEFT JOIN Tickets t
    -> ON b.booking_id = t.booking_id;
+----------------+------------+-----------+-------------+
| passenger_name | booking_id | ticket_id | seat_number |
+----------------+------------+-----------+-------------+
| Rahul Sharma   |          1 |         1 | 12A         |
| Priya Verma    |          2 |         2 | 5B          |
| Amit Singh     |          3 |         3 | 1C          |
+----------------+------------+-----------+-------------+
3 rows in set (0.00 sec)
```

3. Right Join : This query retrieves ticket details along with flight numbers by joining Tickets with Flights

```
mysql> SELECT
    ->     t.ticket_id,
    ->     f.flight_number,
    ->     t.seat_number
    -> FROM Tickets t
    -> RIGHT JOIN Flights f
    -> ON t.flight_id = f.flight_id;
+-----------+---------------+-------------+
| ticket_id | flight_number | seat_number |
+-----------+---------------+-------------+
|         1 | AI101         | 12A         |
|         2 | AI202         | 5B          |
|         3 | AI303         | 1C          |
+-----------+---------------+-------------+
3 rows in set (0.00 sec)
```

4. Subquery (WHERE clause) : This query retrieves the names of passengers who have made at least one booking

```
mysql> SELECT
    ->     name
    -> FROM Passengers
    -> WHERE passenger_id IN (
    ->     SELECT passenger_id
    ->     FROM Bookings
    -> );
+--------------+
| name         |
+--------------+
| Rahul Sharma |
| Priya Verma  |
| Amit Singh   |
+--------------+
3 rows in set (0.01 sec)
```

5. Subquery (Nested) : This query retrieves the names of passengers who have booked tickets for a specific flight (flight_id = 1)

```
mysql> SELECT
    ->     name
    -> FROM Passengers
    -> WHERE passenger_id IN (
    ->     SELECT passenger_id
    ->     FROM Bookings
    ->     WHERE booking_id IN (
    ->         SELECT booking_id
    ->         FROM Tickets
    ->         WHERE flight_id = 1
    ->     )
    -> );
+--------------+
| name         |
+--------------+
| Rahul Sharma |
+--------------+
1 row in set (0.01 sec)
```

6. Subquery (Scalar) : This query retrieves ticket details where the seat number matches the single value returned by the subquery

```
mysql> SELECT
    ->     ticket_id,
    ->     seat_number
    -> FROM Tickets
    -> WHERE seat_number = (
    ->     SELECT seat_number
    ->     FROM Tickets
    ->     LIMIT 1
    -> );
+-----------+-------------+
| ticket_id | seat_number |
+-----------+-------------+
|         1 | 12A         |
+-----------+-------------+
1 row in set (0.01 sec)
```

7. Create a View : This query creates a view named Ticket_Info that combines ticket, passenger, booking, and flight details into a single virtual table.

```
mysql> CREATE VIEW Ticket_Info AS
    -> SELECT
    ->     t.ticket_id,
    ->     p.name AS passenger_name,
    ->     f.flight_number,
    ->     t.seat_number,
    ->     t.class
    -> FROM Tickets t
    -> JOIN Bookings b ON t.booking_id = b.booking_id
    -> JOIN Passengers p ON b.passenger_id = p.passenger_id
    -> JOIN Flights f ON t.flight_id = f.flight_id;
Query OK, 0 rows affected (0.02 sec)
```

8. Querying the View : This query retrieves the names of passengers and their flight numbers from the Ticket_Info view where the ticket class is 'Business'.

```
mysql> SELECT
    ->     passenger_name,
    ->     flight_number
    -> FROM Ticket_Info
    -> WHERE class = 'Business';
+----------------+---------------+
| passenger_name | flight_number |
+----------------+---------------+
| Priya Verma    | AI202         |
+----------------+---------------+
1 row in set (0.01 sec)
```

Part B: GROUP BY and HAVING Clauses

9. GROUP BY : This query retrieves the number of flights originating from each city by grouping data based on city.

```
mysql> SELECT
    ->     a.city,
    ->     COUNT(f.flight_id) AS total_flights
    -> FROM Flights f
    -> JOIN Airports a ON f.source_airport = a.airport_id
    -> GROUP BY a.city;
+-----------+---------------+
| city      | total_flights |
+-----------+---------------+
| Delhi     |             1 |
| Mumbai    |             1 |
| Bangalore |             1 |
+-----------+---------------+
3 rows in set (0.01 sec)
```

10. GROUP BY with HAVING : It uses GROUP BY to group tickets by flight_id and HAVING to filter groups based on count.

```
mysql> SELECT
    ->     flight_id,
    ->     COUNT(*) AS total_tickets
    -> FROM Tickets
    -> GROUP BY flight_id
    -> HAVING COUNT(*) > 2;
Empty set (0.01 sec)

mysql> SELECT
    ->     class,
    ->     COUNT(*) AS total_tickets
    -> FROM Tickets
    -> GROUP BY class
    -> HAVING COUNT(*) > 3;
Empty set (0.00 sec)

mysql> SELECT
    ->     a.city,
    ->     COUNT(f.flight_id) AS total_flights
    -> FROM Flights f
    -> JOIN Airports a ON f.source_airport = a.airport_id
    -> GROUP BY a.city
    -> HAVING COUNT(f.flight_id) > 1;
Empty set (0.00 sec)
```

Mini Project : Civic Voice Interface System

SQL Queries:

Part A: Joins, Subqueries, and Views

1. Inner Join: It returns only those users who have a corresponding profile, showing their personal and profile details together.

```
mysql> SELECT
    ->     u.user_id,
    ->     u.full_name,
    ->     u.email,
    ->     c.address,
    ->     c.date_of_birth
    -> FROM `User` u
    -> INNER JOIN CitizenProfile c
    -> ON u.user_id = c.user_id;
+---------+---------------+-------------------+---------------------+---------------+
| user_id | full_name     | email             | address             | date_of_birth |
+---------+---------------+-------------------+---------------------+---------------+
|       1 | Atharva Patil | atharva@gmail.com | Pune, Maharashtra   | 2003-05-10    |
|       2 | Sneha Joshi   | sneha@gmail.com   | Mumbai, Maharashtra | 2002-08-15    |
|       3 | Rajesh Kumar  | rajesh@gov.in     | Delhi, India        | 1990-01-20    |
+---------+---------------+-------------------+---------------------+---------------+
3 rows in set (0.00 sec)
```

2. Left Join : Users without documents are still included, with NULL values in document fields.

```
mysql> SELECT
    ->     u.full_name,
    ->     d.document_type,
    ->     d.status
    -> FROM `User` u
    -> LEFT JOIN UserDocument d
    -> ON u.user_id = d.user_id;
+---------------+-----------------+----------+
| full_name     | document_type   | status   |
+---------------+-----------------+----------+
| Atharva Patil | Driving License | verified |
| Atharva Patil | Aadhaar         | verified |
| Sneha Joshi   | PAN             | pending  |
| Rajesh Kumar  | NULL            | NULL     |
+---------------+-----------------+----------+
4 rows in set (0.03 sec)
```

3. Right Join: If any profile has no corresponding user, the user fields would appear as NULL (though not in this result).

```
mysql> SELECT
    ->     u.user_id,
    ->     u.full_name,
    ->     c.address,
    ->     c.date_of_birth
    -> FROM `User` u
    -> RIGHT JOIN CitizenProfile c
    -> ON u.user_id = c.user_id;
+---------+---------------+---------------------+---------------+
| user_id | full_name     | address             | date_of_birth |
+---------+---------------+---------------------+---------------+
|       1 | Atharva Patil | Pune, Maharashtra   | 2003-05-10    |
|       2 | Sneha Joshi   | Mumbai, Maharashtra | 2002-08-15    |
|       3 | Rajesh Kumar  | Delhi, India        | 1990-01-20    |
+---------+---------------+---------------------+---------------+
3 rows in set (0.00 sec)
```

4. Subquery (WHERE clause): Find users whose IDs appear in the list of users with verified documents.

```
mysql> SELECT
    ->     full_name,
    ->     email
    -> FROM `User`
    -> WHERE user_id IN (
    ->     SELECT user_id
    ->     FROM UserDocument
    ->     WHERE status = 'verified'
    -> );
+---------------+-------------------+
| full_name     | email             |
+---------------+-------------------+
| Atharva Patil | atharva@gmail.com |
+---------------+-------------------+
1 row in set (0.01 sec)
```

5. Subquery (Nested): This query uses a **nested subquery** to find departments that offer services with processing time greater than 10 days.

```
mysql> SELECT department_name
    -> FROM Department
    -> WHERE department_id IN (
    ->     SELECT department_id
    ->     FROM GovernmentService
    ->     WHERE processing_time IN (
    ->         SELECT processing_time
    ->         FROM GovernmentService
    ->         WHERE processing_time > '10 days'
    ->     )
    -> );
+-----------------+
| department_name |
+-----------------+
| Water Supply    |
| Electricity     |
| Transport       |
+-----------------+
3 rows in set (0.01 sec)
```

6. Create a View: This query creates a **VIEW** named User_Profile_View that combines user and profile data using a LEFT JOIN.

```
mysql> CREATE VIEW User_Profile_View AS
    -> SELECT
    ->     u.user_id,
    ->     u.full_name,
    ->     u.email,
    ->     c.address,
    ->     c.date_of_birth
    -> FROM `User` u
    -> LEFT JOIN CitizenProfile c
    -> ON u.user_id = c.user_id;
Query OK, 0 rows affected (0.11 sec)
```

7. Querying the View: This query retrieves data from the **User_Profile_View** and filters only those users who have a non-null address.

```
mysql> SELECT full_name, address
    -> FROM User_Profile_View
    -> WHERE address IS NOT NULL;
+---------------+---------------------+
| full_name     | address             |
+---------------+---------------------+
| Atharva Patil | Pune, Maharashtra   |
| Sneha Joshi   | Mumbai, Maharashtra |
| Rajesh Kumar  | Delhi, India        |
+---------------+---------------------+
3 rows in set (0.03 sec)
```

8. Multiple Joins: This query uses **multiple LEFT JOINs** to combine User, CitizenProfile, and UserDocument into a single result.

```
mysql> SELECT
    ->     u.full_name,
    ->     u.email,
    ->     c.address,
    ->     d.document_type,
    ->     d.status
    -> FROM `User` u
    -> LEFT JOIN CitizenProfile c
    ->     ON u.user_id = c.user_id
    -> LEFT JOIN UserDocument d
    ->     ON u.user_id = d.user_id;
+---------------+-------------------+---------------------+-----------------+----------+
| full_name     | email             | address             | document_type   | status   |
+---------------+-------------------+---------------------+-----------------+----------+
| Atharva Patil | atharva@gmail.com | Pune, Maharashtra   | Driving License | verified |
| Atharva Patil | atharva@gmail.com | Pune, Maharashtra   | Aadhaar         | verified |
| Sneha Joshi   | sneha@gmail.com   | Mumbai, Maharashtra | PAN             | pending  |
| Rajesh Kumar  | rajesh@gov.in     | Delhi, India        | NULL            | NULL     |
+---------------+-------------------+---------------------+-----------------+----------+
4 rows in set (0.00 sec)
```

9. Subquery with EXISTS: This query uses a **subquery with EXISTS** to check if a user has at least one related document.

```
mysql> SELECT
    ->     u.full_name,
    ->     u.email
    -> FROM `User` u
    -> WHERE EXISTS (
    ->     SELECT 1
    ->     FROM UserDocument d
    ->     WHERE d.user_id = u.user_id
    -> );
+---------------+-------------------+
| full_name     | email             |
+---------------+-------------------+
| Atharva Patil | atharva@gmail.com |
| Sneha Joshi   | sneha@gmail.com   |
+---------------+-------------------+
2 rows in set (0.01 sec)
```

Part B: GROUP BY and HAVING Clauses

10. GROUP BY : This query uses **LEFT JOIN with GROUP BY** to count the number of documents associated with each user.

```
mysql> SELECT
    ->     u.full_name,
    ->     COUNT(d.document_id) AS total_documents
    -> FROM `User` u
    -> LEFT JOIN UserDocument d
    ->     ON u.user_id = d.user_id
    -> GROUP BY u.user_id, u.full_name;
+---------------+-----------------+
| full_name     | total_documents |
+---------------+-----------------+
| Atharva Patil |               2 |
| Sneha Joshi   |               1 |
| Rajesh Kumar  |               0 |
+---------------+-----------------+
3 rows in set (0.01 sec)
```

11. GROUP BY with HAVING: This query uses **GROUP BY with HAVING** to filter users based on aggregated document count.

```
mysql> SELECT
    ->     u.full_name,
    ->     COUNT(d.document_id) AS total_documents
    -> FROM `User` u
    -> LEFT JOIN UserDocument d
    ->     ON u.user_id = d.user_id
    -> GROUP BY u.user_id, u.full_name
    -> HAVING COUNT(d.document_id) > 1;
+---------------+-----------------+
| full_name     | total_documents |
+---------------+-----------------+
| Atharva Patil |               2 |
+---------------+-----------------+
1 row in set (0.00 sec)
```

12. GROUP BY and Joins: This query uses **JOINs with GROUP BY** to count the number of documents associated with each gender.

```
mysql> SELECT
    ->     c.gender,
    ->     COUNT(d.document_id) AS total_documents
    -> FROM CitizenProfile c
    -> INNER JOIN `User` u
    ->     ON c.user_id = u.user_id
    -> LEFT JOIN UserDocument d
    ->     ON u.user_id = d.user_id
    -> GROUP BY c.gender;
+--------+-----------------+
| gender | total_documents |
+--------+-----------------+
| male   |               2 |
| female |               1 |
+--------+-----------------+
2 rows in set (0.00 sec)
```

13. GROUP BY with HAVING and Joins: This query uses **JOIN + WHERE + GROUP BY + HAVING** to count verified documents per user.

```
mysql> SELECT
    ->     u.full_name,
    ->     COUNT(d.document_id) AS verified_docs
    -> FROM `User` u
    -> INNER JOIN UserDocument d
    ->     ON u.user_id = d.user_id
    -> WHERE d.status = 'verified'
    -> GROUP BY u.user_id, u.full_name
    -> HAVING COUNT(d.document_id) > 1;
+---------------+---------------+
| full_name     | verified_docs |
+---------------+---------------+
| Atharva Patil |             2 |
+---------------+---------------+
1 row in set (0.00 sec)
```